

R4.02 - Qualité de développement 3

Conception de tests

Jean-Marie Mottu
BUT info 2 – IUT Nantes

Organisation de R4.02 - QD3

▶ Volume

- ▶ 3 cours magistraux
- ▶ 5 TD de 2h40

▶ Enseignants

- ▶ Jean-Marie Mottu, responsable
- ▶ Sébastien Fauvel, Emmanuel Bigeon

▶ Notation en contrôle continu :

- ▶ QCM en amphi
- ▶ Note(s) pratique(s) pendant les séances
- ▶ Contrôle pratique sur machine en fin de période

Plan dans le BUT

- ▶ Une des quatre ressources de Qualité de Développement :
 - ▶ R2.03 – QDI Automatisation des tests
 - ▶ R3.04 – QD2 Conception avancée
 - ▶ R4.02 – QD3 Conception de tests
 - ▶ R5.A.08 – QD4 Conception et Test dans le Cycle de Développement

- ▶ Lien direct avec les ressources
 - ▶ R2.01 - Développement orienté objets
 - ▶ R2.10 – R3.10 Gestion de Projet Informatique

Plan de R4.02 - Qualité de développement 3

Conception de tests

- ▶ Rappel de QDI – Test Logiciel (CMI)
 - ▶ Rappel de Typologie
 - ▶ Rappel du Test Fonctionnel –
Injection de Dépendance – Doublure (TDI)
- ▶ « Couverture de tests » :
 - ▶ Test structurel (CM2 – TD2)
 - ▶ Analyse de mutation (CM3 – TD3)
 - ▶ Synthèse (TD4)
- ▶ Test d'intégration (CM4 – TD5)

Enjeux de contrôler la qualité logicielle

- ▶ Tester pour détecter les fautes en provoquant des erreurs avant qu'elles deviennent des défaillances
- ▶ Des bugs, des bugs, des bugs
 - ▶ 19/07/2024 : panne mondiale (8,5 millions de systèmes Windows) suite à une mise à jour défectueuse du capteur de l'agent Falcon de CrowdStrike :
 - ▶ <https://www.lemondeinformatique.fr/actualites/lire-panne-geante-crowdstrike-reconnait-un-manque-de-tests-94375.html>
 - ▶ <https://www.lemondeinformatique.fr/actualites/lire-une-entree-de-trop-a-l-origine-du-bug-de-crowdstrike-94462.html>
- ▶ CPAM – migration de données (2024-2025)
 - ▶ Coût pour l'usager : des millions d'euros d'indemnités journalières à des milliers d'utilisateurs non versés

Tester en réalisant un ensemble fini de cas de test

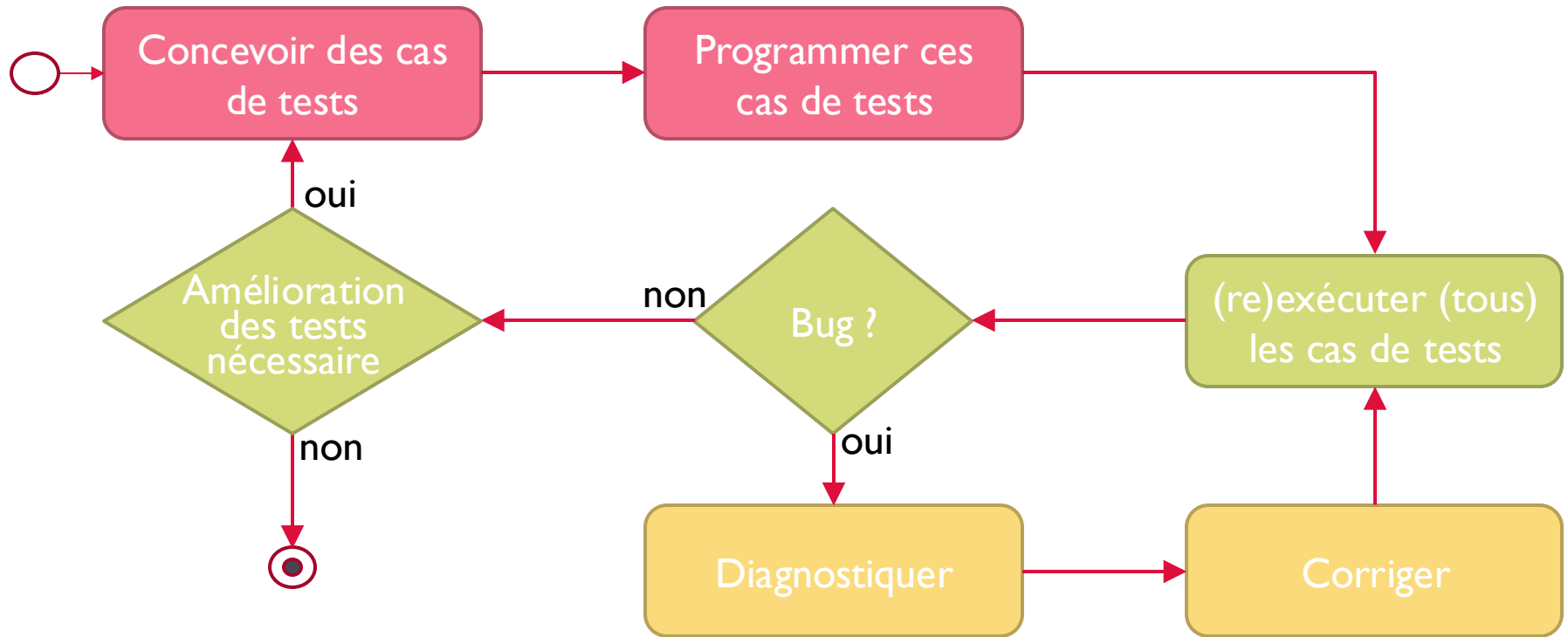
▶ Cas de test

- ▶ Description du cas de test
- ▶ Initialisation du SUT (System Under Test)
- ▶ Donnée de test (DT)
- ▶ Oracle
 - ▶ L'Oracle contrôle l'exécution du SUT initialisé, avec la DT, retournant le verdict
- ▶ Verdict : passe, échoue
- ▶ Sur la base d'une spécification
- ▶ Le tout devant être analysé, conçu, automatisé*, diagnostiqué

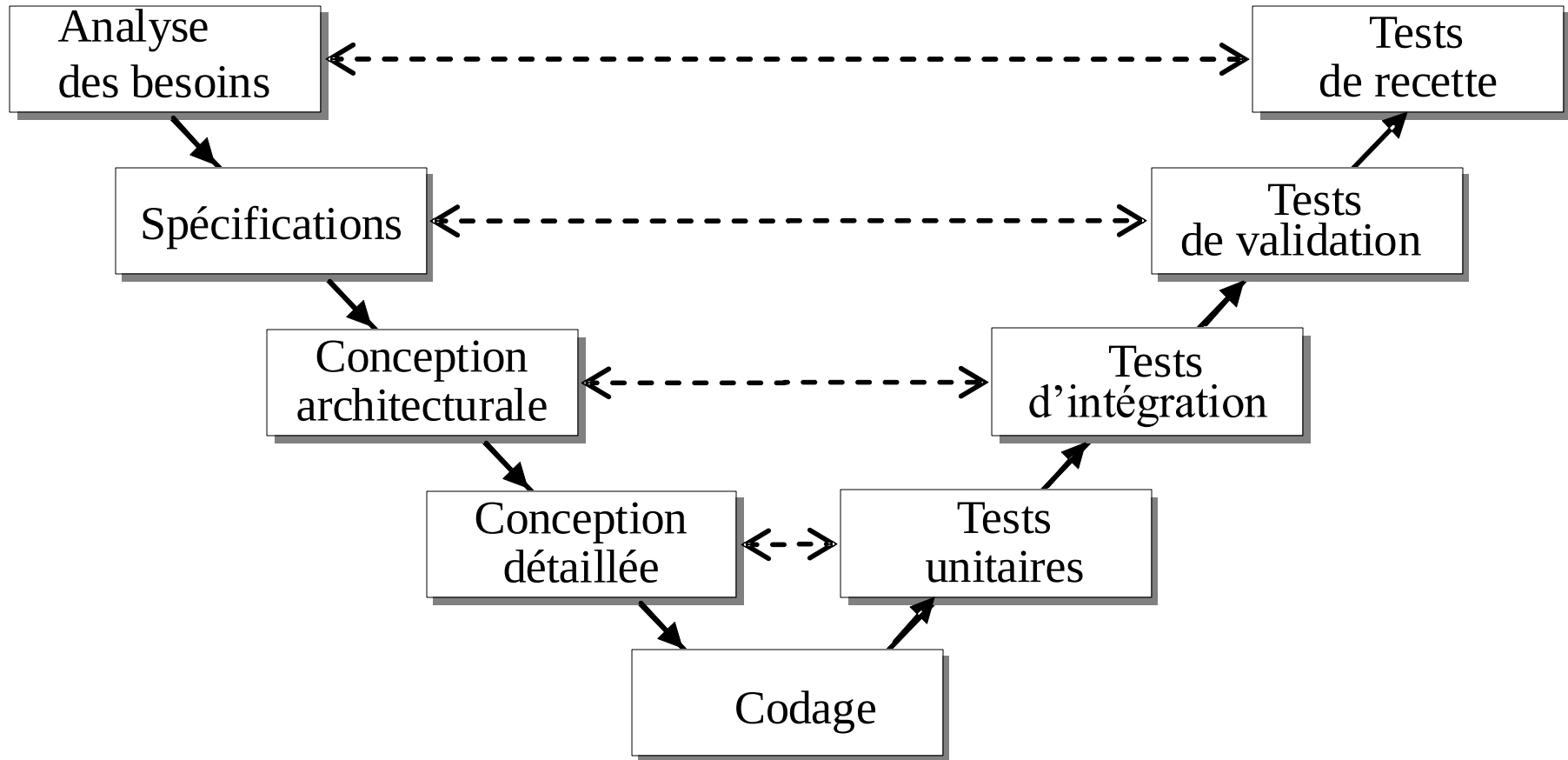
Test de logiciel : terminologie

- ▶ **Etapes de test**
 - ▶ Unitaire, intégration, système, recette
- ▶ **Phases transversales**
 - ▶ non régression, performance
- ▶ **Différente dynamicité, les tests peuvent être**
 - ▶ Dynamique / statique
- ▶ **Test du comportement**
 - ▶ Nominal / Exceptionnel
- ▶ **Création de tests avec une approche**
 - ▶ Fonctionnelle / structurelle
 - ▶ Chacune ayant plusieurs techniques de création/sélection de cas de test

Le cycle de test dynamique



Le test dans un cycle de développement en V

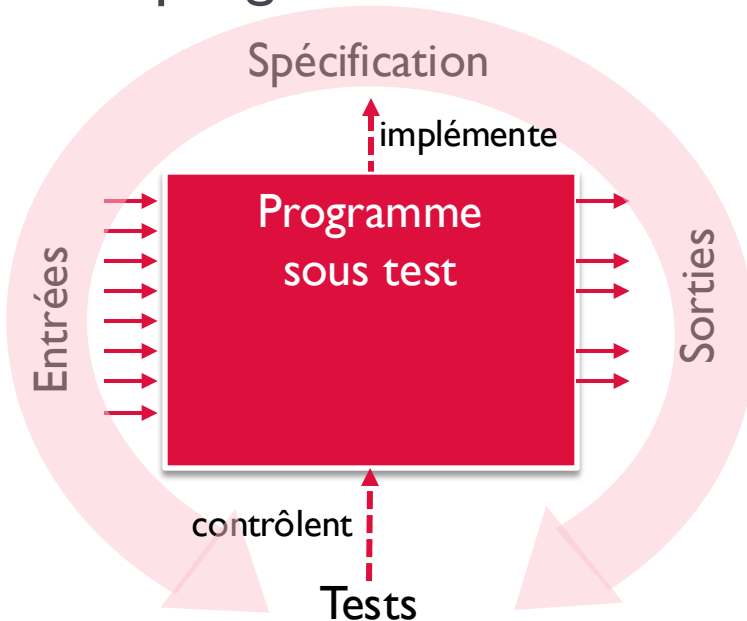


Couverture des tests :

Création de tests avec différentes approches

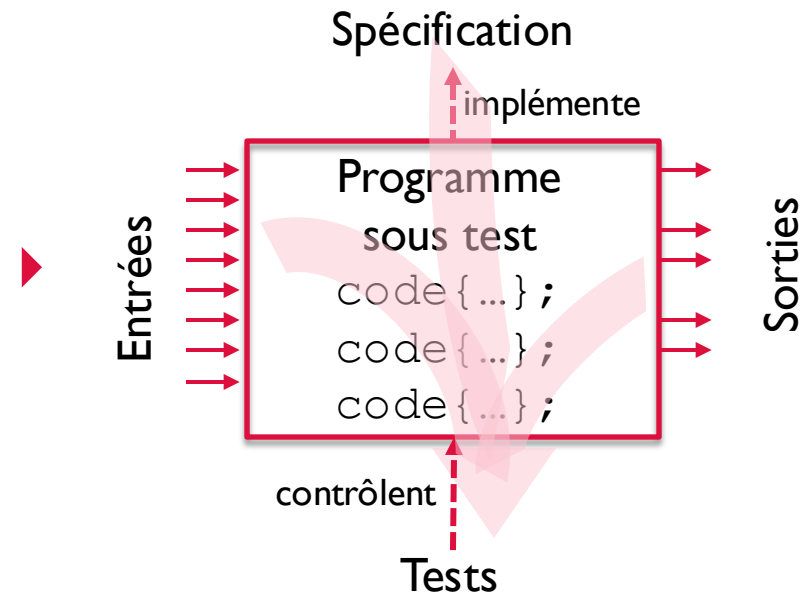
► Test fonctionnel (test boîte noire)

- Exploite la description des fonctionnalités du programme

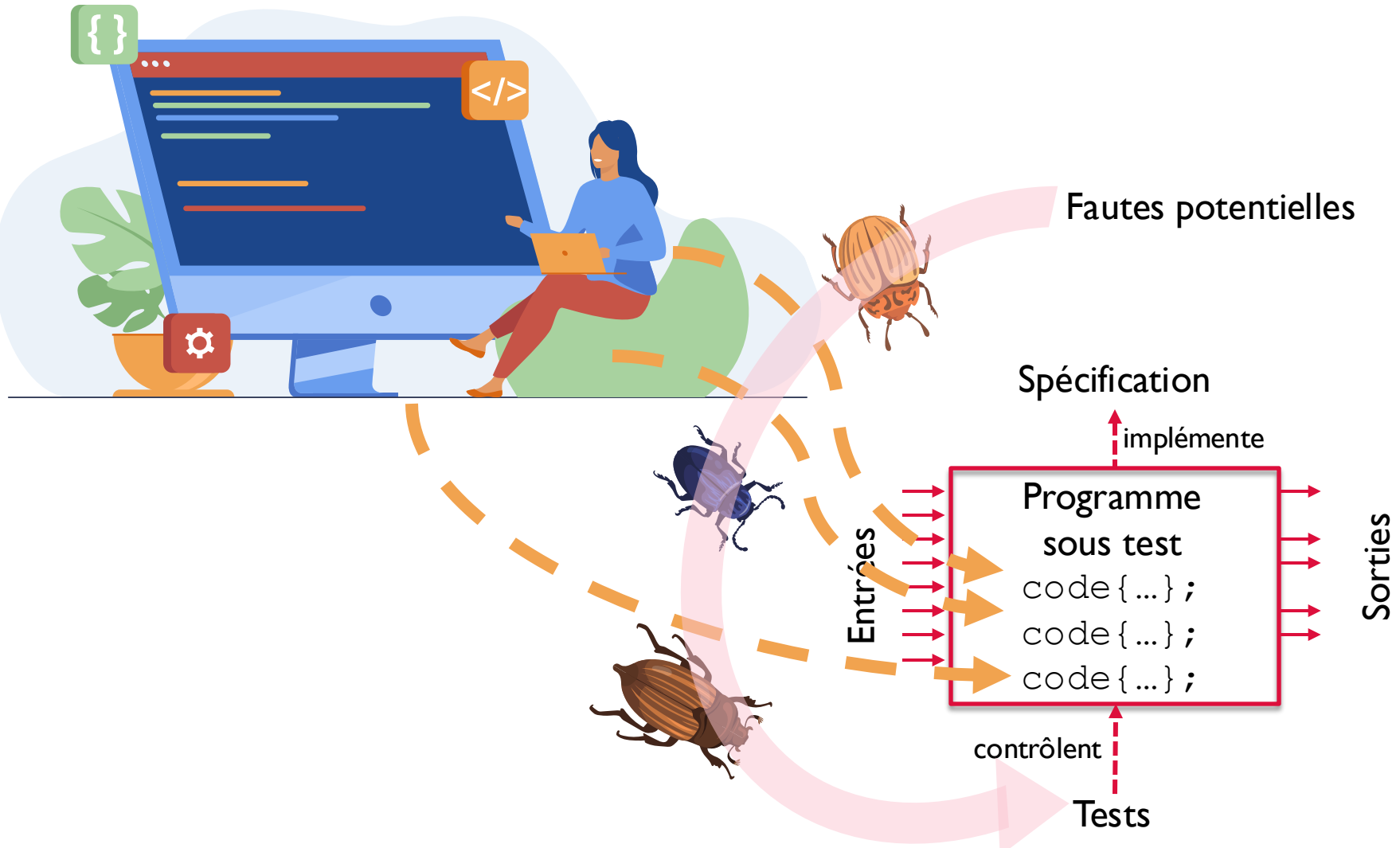


► Test structurel (test boîte blanche)

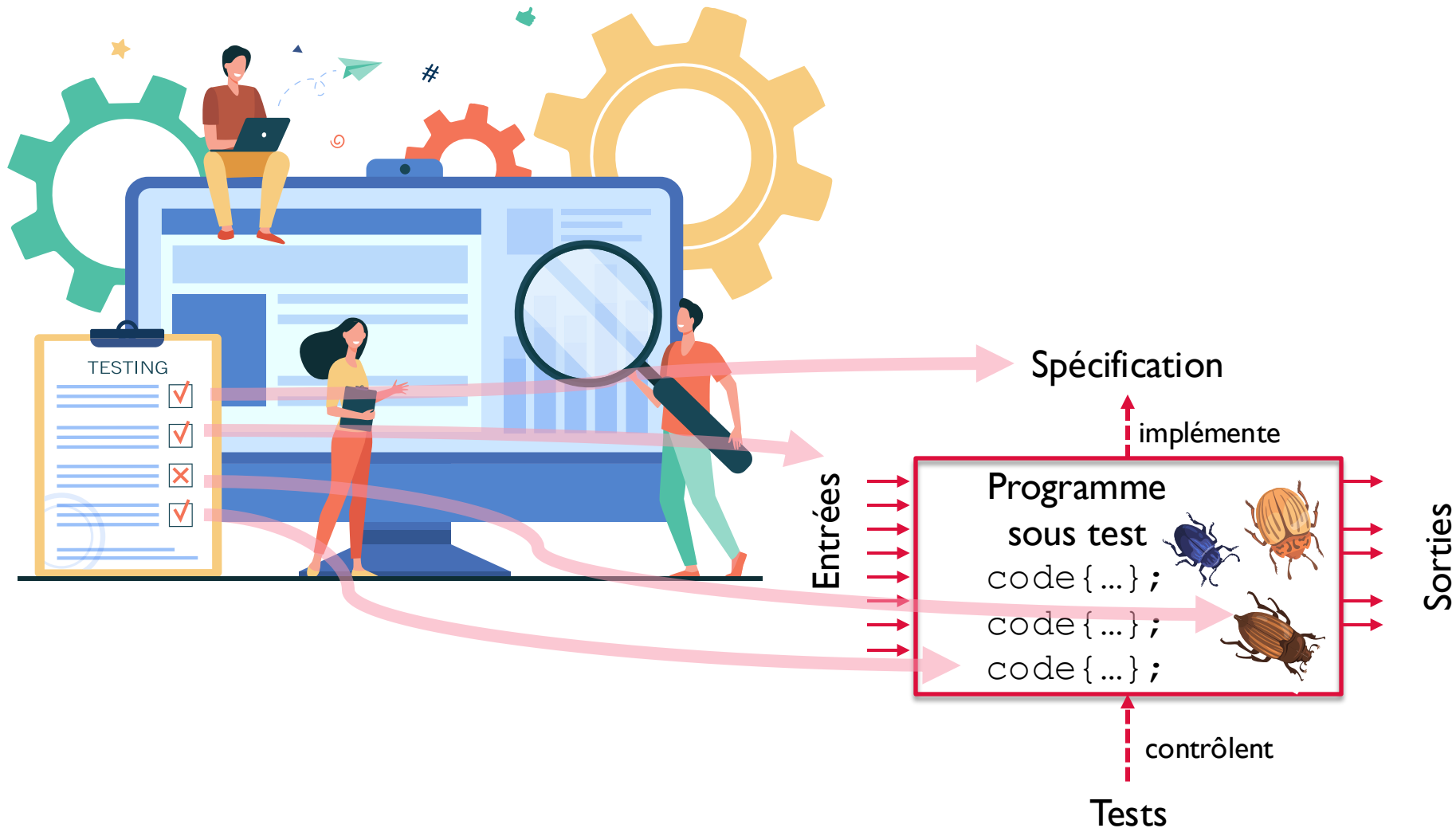
- Exploite la structure interne du programme



Couverture des tests : L'analyse de mutation pour couvrir les fautes potentielles



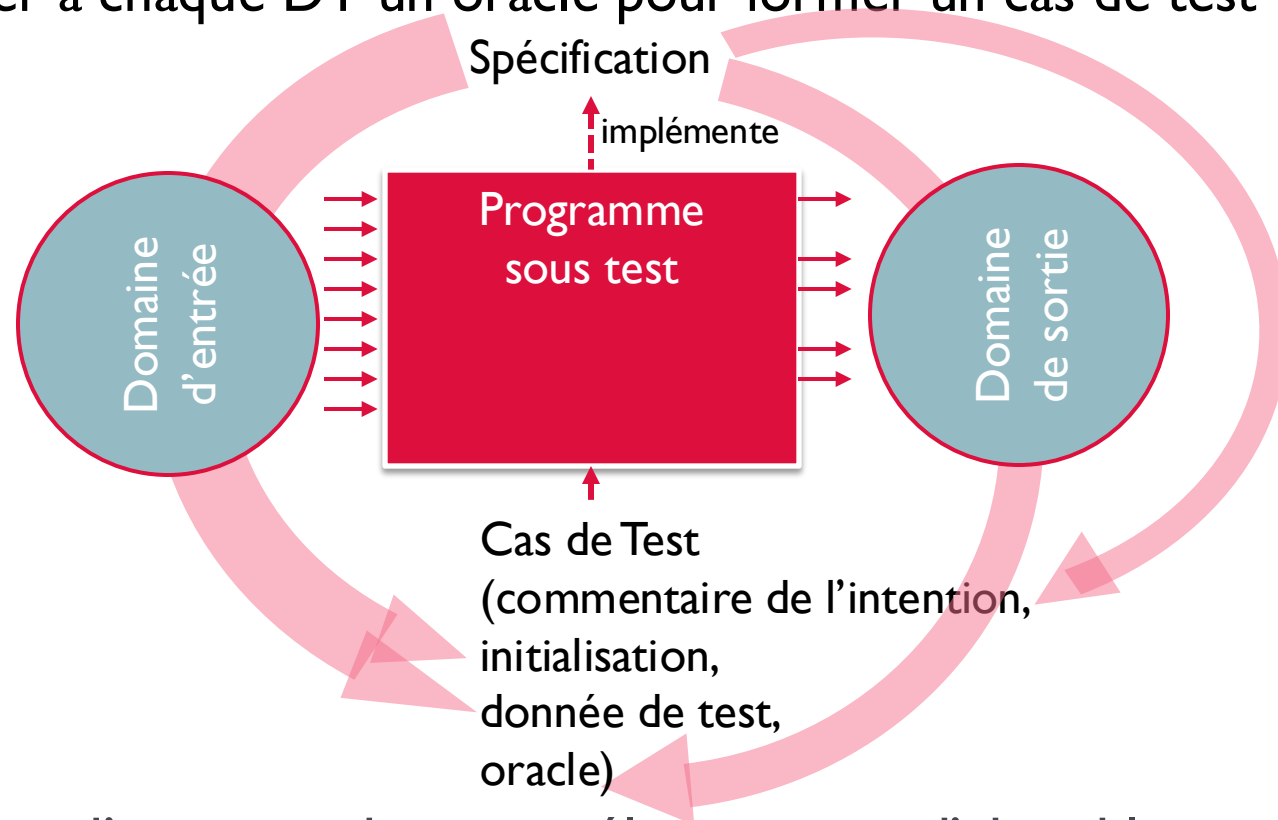
Couverture des tests : Spécification / Code / Fautes Potentielles



Conception de test par approche fonctionnelle Synthèse de l'analyse partitionnelle

Objectif : sélectionner des données de test dans l'espace de données par approche fonctionnelle

- ▶ Guidé la conception des cas de test par une sélection des **données de test** :
 - ▶ En nombre fini choisies pour leur qualité, leur représentativité, etc.
 - ▶ Associer à chaque DT un oracle pour former un cas de test



- ▶ Remarque : d'autres techniques sélectionnent d'abord les oracles

Méthode de l'analyse partitionnelle

► Pour chaque variable formant chaque donnée de test

1. identifier le type de la variable (int sur 32bits par exemple)
2. identifier la plage de la variable
 - A. Des intervalles nominaux
 - La plage de valeurs du fonctionnement normal (les mois de 1 à 12)
 - B. Un/des intervalles exceptionnels
 - La/les plages de valeurs du fonctionnement exceptionnel (<1 et >12)
 - Puisque le type permet de passer ces valeurs, il faut les tester

3. **identifier des partitions fonctionnelles**

- A. **Anticipe généralement les combinaisons des variables**
- B. **Etape difficile nécessitant une maîtrise de la spécification**
4. choisir une valeur dans chaque intervalle et combiner pour former les données de test

Table de décision pour gérer plus de 2 dimensions

Exemple du nombre de jour

Entrees	mois	$[-2^3 ,]$	X									
		{1, 3, 5, 7, 8, 10, 12}					X	X				
		{4, 6, 9, 11}							X	X		
		2									X	X
		$] 2, 2^3 -]$		X								
	annee	$[-2^3 , 200]$			X							
		AnneesBissextiles					X		X		X	
		AutresAnnees						X		X		X
		$] 2 00, 2^3 -]$				X						
Sortie attendue	31						X	X				
	30								X	X		
	29										X	
	28											X
	entrees invalides		X	X	X	X						

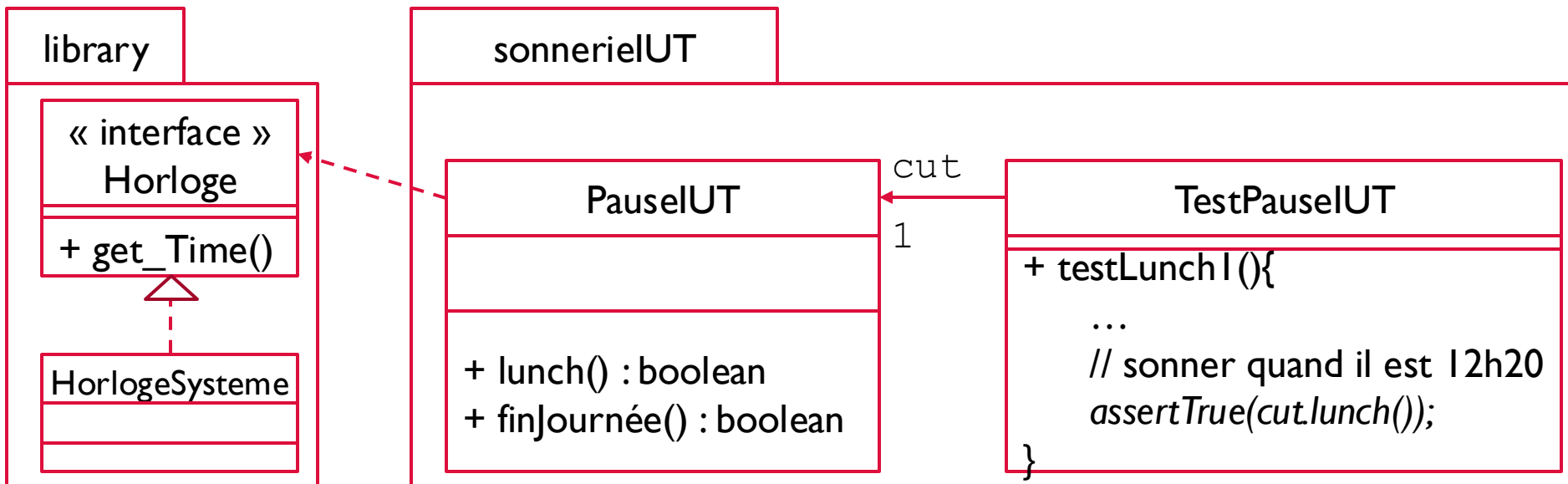
Testabilité et gestion de dépendances

Problème de testabilité à gérer pendant les tests

- ▶ Comment tester efficacement une unité qui dépend d'une autre qui :
 - ▶ n'existe pas encore
 - ▶ n'est pas contrôlable
 - ▶ Le temps, l'aléatoire, l'indéterminisme
 - ▶ Des liaisons vers le monde physique
 - ▶ n'est pas fiable
 - ▶ e.g. impliquée dans des cycles d'interdépendances
 - ▶ est difficilement configurable
 - ▶ a des effets de bords
 - ▶ est lente ou coûteuse
 - ▶ Des conditions extrêmes ou exceptionnelles

Problème de testabilité à gérer pendant les tests

- ▶ Comment tester efficacement une unité qui dépend d'une autre qui :
 - ▶ n'existe pas encore
 - ▶ n'est pas contrôlable



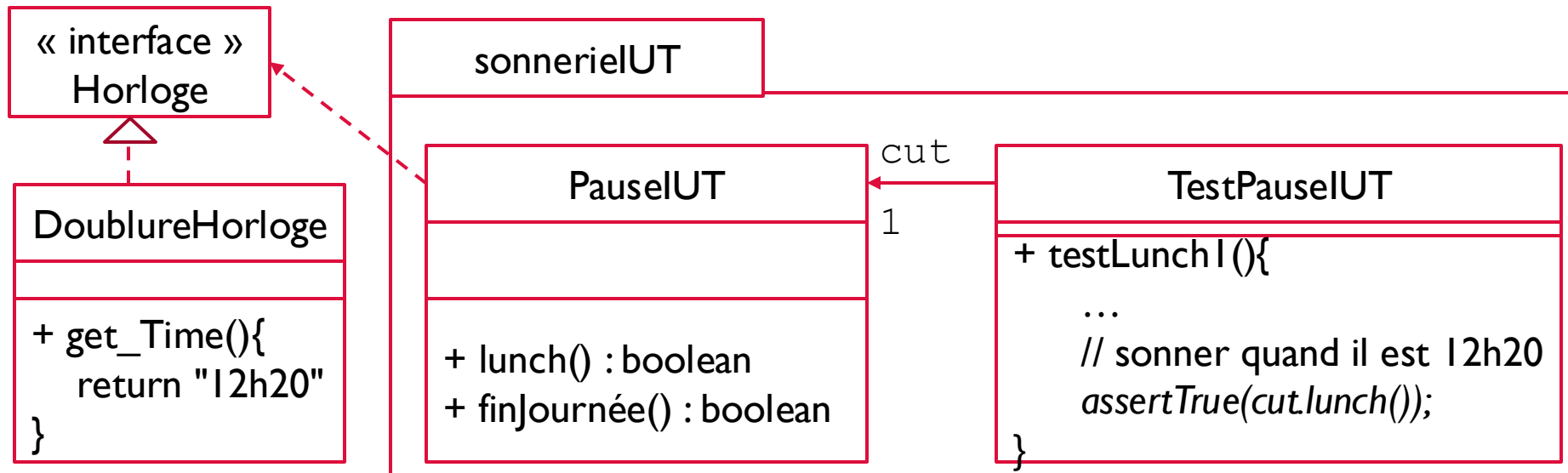
Solution : doublure de test

- ▶ Remplacer l'unité dont dépend l'unité sous test par un équivalent spécifique au test : sa “doublure”
- ▶ Différentes variantes de doublures sont possibles.
- ▶ Indépendamment de la variante, seul le comportement nécessaire pour tester est implémenté.

Solution :

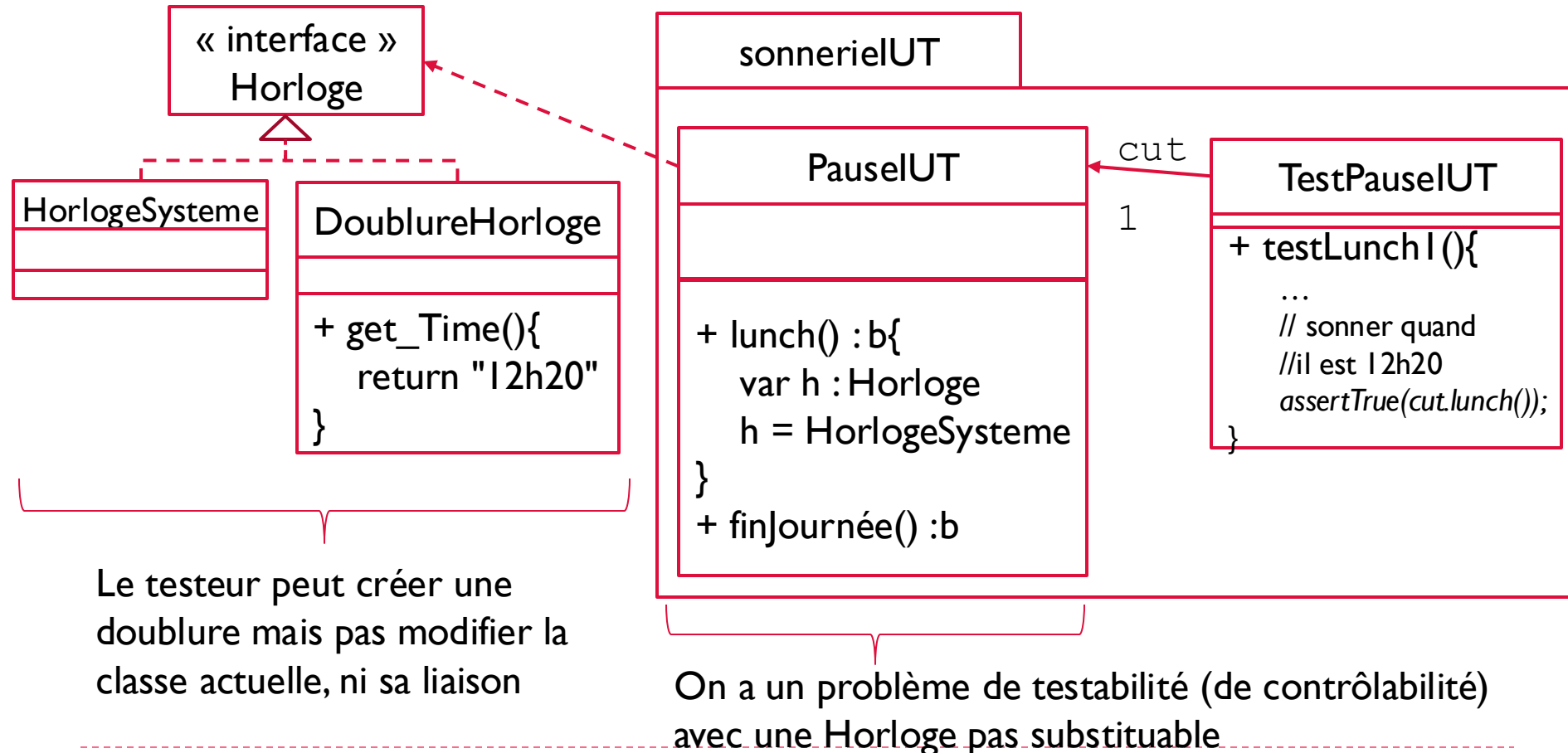
doublure de test

- ▶ Remplacer l'unité dont dépend l'unité sous test par un équivalent spécifique au test : sa "doublure"
- ▶ Différentes variantes de doublures sont possibles.
- ▶ Indépendamment de la variante, seul le comportement nécessaire pour tester est implémenté.



Comment gérer la substitution de la dépendance ?

- Règle : le testeur ne peut pas modifier (temporairement) le code sous test



Injection de dépendance

- ▶ Créer une doublure nécessite de pouvoir la substituer à la classe originale.
- ▶ Le principe est de ne pas permettre à une méthode de gérer ses dépendances
 - ▶ Sinon le testeur devrait modifier son code pour utiliser la doublure
- ▶ Les dépendances d'une classe doivent lui être fournies depuis l'extérieur
 - ▶ Basiquement en paramètre du constructeur
 - ▶ En limitant les instantiations dans les méthodes (new...)

Injection de dépendance : contre-exemple

```
interface Horloge {
    int getHour();
    int getMin();
}

public class PauseIUT{

    PauseIUT(){

    }

    boolean lunch(){
        // dépendance pas substituable
        Horloge horlogeActual = new HorlogeSysteme();
        return horlogeActual.getHour() == 12
            &&
            horlogeActual.getMin() == 20;
    }
}
```


Injection de dépendance de classe : bonne pratique

```
interface Horloge {  
    int getHour();  
    int getMin();  
}  
  
public class PauseIUT{  
    // dépendance substituable  
    Horloge horlogeUsed;  
    PauseIUT(Horloge horlogeParam) {  
        horlogeUsed = horlogeParam;  
    }  
  
    boolean lunch() {  
        return horlogeUsed.getHour() == 12  
            &&  
            horlogeUsed.getMin() == 20;  
    }  
}
```

Injection de dépendance de classe : bonne pratique

```
public class testPauseIUT{  
    boolean testlunch1(){  
        Horloge hDoublure = new HorlogeDoublure()  
        // HorlogeDoublure renvoie tout le temps 12h20  
        PauseIUT cut = new PauseIUT(hDoublure)  
        assertTrue(cut.lunch, "A 12h20 c'est la pause")  
    }  
}
```

